📘 Understanding the Whole Project

A simple explanation of what this project is, how it is built, and how it works — no tech background needed.

1. What is this project, really?

It is a website for a university office (a "registrar"). It has two jobs:

1. **Manage records** — students, departments, courses, and who is enrolled in what. This part already existed.
2. **Answer questions** — a chat assistant called **UniAssist AI** that students can ask things like "What are the tuition fees?" This part was just added.

💡 Think of it like a school office with a filing cabinet (records) and, right next to it, a help desk with a smart assistant (the chat) who has read every policy document in the cabinet.

---

2. The two big halves of the project

Backend

The "brain" — Python / FastAPI

Lives in the `backend` folder. It does the real work: talks to the database, checks rules, and (for UniAssist AI) finds answers and asks the AI to write them.

Nobody sees this directly — it quietly answers requests sent to it.

Frontend

The "face" — React website

Lives in the `frontend` folder. It's what you actually see and click in your browser — pages, buttons, forms, and the chat box.

It never talks to the database directly. It always asks the backend.

How they talk to each other: the frontend sends a message over the internet (even if it's just your own computer) to the backend, the backend does the work, and sends an answer back. This is called an **API** — just a fancy word for "a way for two programs to talk."

3. The original part: Students, Courses, Departments

This part was already built before UniAssist AI was added. It follows a simple, repeated pattern for every kind of record (students, courses, departments, enrollments):

1. **Route** — receives the request (e.g. "add a new student")
2. **Schema** — checks the data looks correct (e.g. name isn't empty)
3. **Service** — does the actual logic (e.g. "check this student doesn't already exist")
4. **Model** — describes how it's stored in the database
5. **Database** — a file called `university.db` that holds everything

Every page on the site (Students, Courses, Departments, Enrollments) follows this exact same pattern — once you understand one, you understand them all.

4. The new part: UniAssist AI — how does a chat assistant "know" things?

UniAssist AI does **not** use ChatGPT's general knowledge of the world. It only knows what is written in your university's own documents (text files about fees, admissions, policies, etc).

> 💡 Imagine a new staff member who has memorized every page of your school's policy handbook, but knows nothing else about the outside world. Ask them about tuition fees — great answer. Ask them about the weather in Paris — they say "that's not in my handbook."

Step by step: what happens when a student asks a question

1. Student types a question in the chat box on the website
2. The question is sent to the backend

3 The backend searches the documents **two different ways at once** (explained below)

4 It combines both search results into one best list

5 It picks the 5 most relevant pieces of text

6 It gives those 5 pieces to the AI model and says "answer using ONLY this"

7 The AI writes an answer, and it's sent back to the student, along with which documents it came from

5. The two ways it searches — explained simply

Search 1

Meaning search (Dense)

This search understands the **meaning** of your question, not just the exact words. If you ask "How much does it cost to study here?", it can find a document about "tuition fees" even though you never used the word "fees."

Tools used: **BGE-M3** (reads the meaning) + **Qdrant** (stores and searches by meaning).

Search 2

Keyword search (Sparse)

This search looks for the **exact words** you typed, like an old-school search engine. It's great for exact names, codes, or specific terms that the meaning search might miss.

Tool used: **BM25** (a well-known keyword search method).

Combining the two (RRF)

Each search makes its own list of "best matching" document pieces. A method called **RRF** (Reciprocal Rank Fusion) merges both lists into one fair ranking — anything that both searches agreed was good gets ranked even higher. Think of it like two different judges scoring a contest, and you combine their scorecards into one final result.

6. Where the documents live, and how the AI "learns" them

Your `.txt` documents sit in `backend/data/documents/`. On their own, the AI can't search them instantly — they first need to be **indexed** (prepared for fast searching). This happens when you run:

```
python scripts/rebuild_index.py
```

This one command does 4 things:

1. Reads every `.txt` file in the documents folder
2. Cuts each one into small chunks (about a paragraph each)
3. Turns each chunk into a "meaning fingerprint" and saves it in Qdrant
4. Also builds the keyword (BM25) search index from the same chunks

You only need to run this again if you add or change a document. Normal chatting never changes anything — it only reads.

7. Who actually writes the answer?

The final writing step is done by an AI language model, reached through a service called **OpenRouter**. The backend sends it the student's question plus the 5 relevant document chunks, and tells it clearly:

"Only answer using this information. If it's not here, say you don't know."

This is why UniAssist AI won't make up fake policies — it's instructed to stick to the real documents, and if nothing relevant was found, it says so honestly instead of guessing.

Your API key for this service is kept only on the backend — the website (frontend) never sees it, so it can't be stolen by someone looking at the webpage.

8. A map of the important folders

FOLDER / FILE	WHAT IT'S FOR
`backend/app/models/`	Describes how records (students, courses...) are stored
`backend/app/routes/`	The "doors" the frontend knocks on to ask for something
`backend/app/services/`	The actual logic/rules for each feature

`backend/app/rag/`	All of UniAssist AI's search & answer logic
`backend/data/documents/`	Your university's `.txt` knowledge files go here
`backend/data/bm25/`	The saved keyword-search index (auto-created)
`backend/scripts/rebuild_index.py`	Run this to teach the AI your documents
`backend/.env`	Your secret settings (API key, etc) — never share this file
`frontend/src/pages/`	Each page you see on the website
`frontend/src/components/`	Reusable pieces (buttons, forms, the chat box)
`frontend/src/api/`	Code that sends requests to the backend

9. Simple glossary

WORD	IN PLAIN ENGLISH
API	A way for two programs to talk to each other
Backend	The hidden part that does the real work
Frontend	The part you see and click in the browser
Database	Where records are permanently stored
RAG	"Retrieval-Augmented Generation" — look things up, then write an answer using what you found
Chunk	A small piece of a document (about a paragraph)
Embedding / BGE-M3	A way to turn text into a "meaning fingerprint" a computer can compare
Vector database / Qdrant	A storage system built for searching by meaning
BM25	A classic keyword-matching search method
RRF	A fair way to combine two different search results into one
Index / Indexing	Preparing documents so they can be searched instantly
LLM	"Large Language Model" — the AI that writes the final answer

OpenRouter	The service that connects to the AI model

10. The big picture, in one line

Old part: a website for managing student/course records, stored in a database.
New part: a chat assistant that searches your school's documents two smart ways, combines the results, and asks an AI to write an honest answer using only what it found.